CSC490 Assignment A3: Pre-training nanochat

Prithvi Seran (1009973343) Vraj Shah (1009102695) Tyseer Toufiq (1009364064)
Joshua Crisologo (1009860438)

Part 1: Architecture Review

The Original nanochat (October 13, 2025)

The original nanochat [1] (commit `3a5e0bc`) is a decoder-only, pre-norm Transformer [2, 3]. Its size is set by the depth L alone: the width is $d = 64L$, each head has 128 dimensions, and the MLP hidden size is $4d$. The d20 model used for the speedrun (Figure 1) has about 561M parameters. Table 1 lists its main design choices.

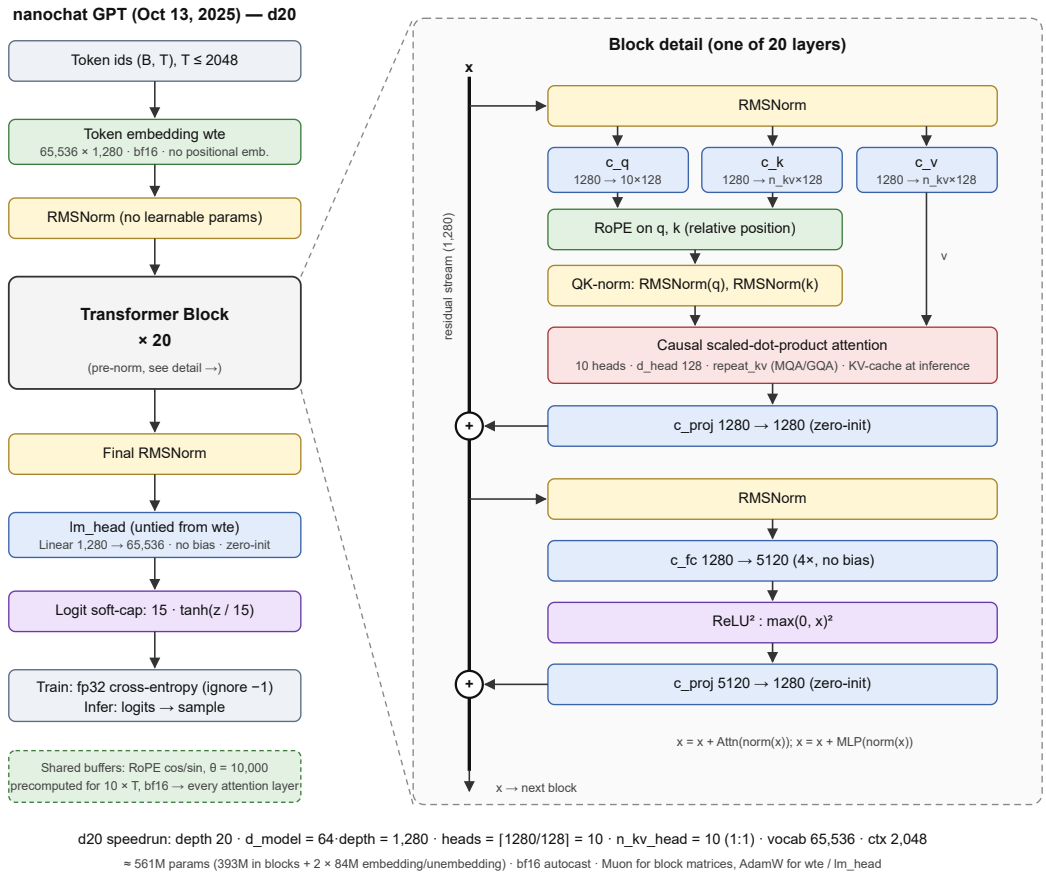


Figure 1: The original nanochat architecture (October 13, 2025) at depth 20. Left: the full model. Right: one of the 20 pre-norm Transformer blocks.

Table 1: Main architecture choices in the original nanochat.

Component	Choice	Why
Position	RoPE on q and k, $\theta = 10{,}000$, all head dims rotated [4]	Relative position with no extra parameters

Component	Choice	Why
Normalization	RMSNorm with no learnable gain, pre-norm, also after the embedding [5]	Cheaper than LayerNorm and keeps the residual stream stable
Attention	Causal multi-head attention with QK-norm [6]	QK-norm bounds attention logits at high learning rates
MLP	$4d$ hidden, ReLU^2 activation [7]	Same cost as GELU and strong in speedruns
Output	Untied `lm_head`; logit soft-cap $15\tanh(z/15)$ [8]	Separate learning rates; prevents over-confident logits
Other	No biases; output projections initialized to zero	Simpler; each block starts as the identity
Optimizer	Muon [9] for the block matrices, AdamW [10] for the embedding and `lm_head`	Muon trains the large matrices faster

Changes in the July 2026 Version

Our baseline is the July 2026 version (`92d63d4`). Most of the changes since October come from the modded-nanogpt speedrun [11]:

- sliding-window attention in an `SSSL` pattern, using FlashAttention-3 [12];
- value embeddings [13] and learned per-layer residual scalars;
- RoPE θ raised to 100,000 and a smaller vocabulary of 32,768 tokens;
- one combined Muon/AdamW optimizer, now with weight decay.

The ReLU^2 MLP has not changed, and RoPE is still applied to every head dimension with one θ for all layers. Our proposed changes target these two parts.

Proposed Changes

ModernBERT [14] updated BERT [15] with components from recent LLMs and tested its choices with ablations. July nanochat already has most of them, including bias-free layers, RoPE, pre-norm, alternating local and global attention, and FlashAttention. We take three ideas from ModernBERT that nanochat lacks: the GeGLU MLP, separate RoPE θ values for local and global layers, and partial RoPE, which ModernBERT tested but did not adopt. Table 2 summarizes them and Figure 2 shows each one.

What we ablate. Part 2 ablates GeGLU and partial RoPE. At d12 neither changes the parameter count or FLOPs, and each is a single flag, so a gap from the baseline that is larger than seed noise can be attributed to the change. We did not test change 3. ModernBERT found no difference between θ settings during pretraining and only raised the global θ when it extended its context length, so the change could only show an effect in a context-extension run like Part 4's, which we ran without it. ModernBERT's ablations used encoders of at most 150M parameters, so the results may not carry over to a causal decoder like nanochat.

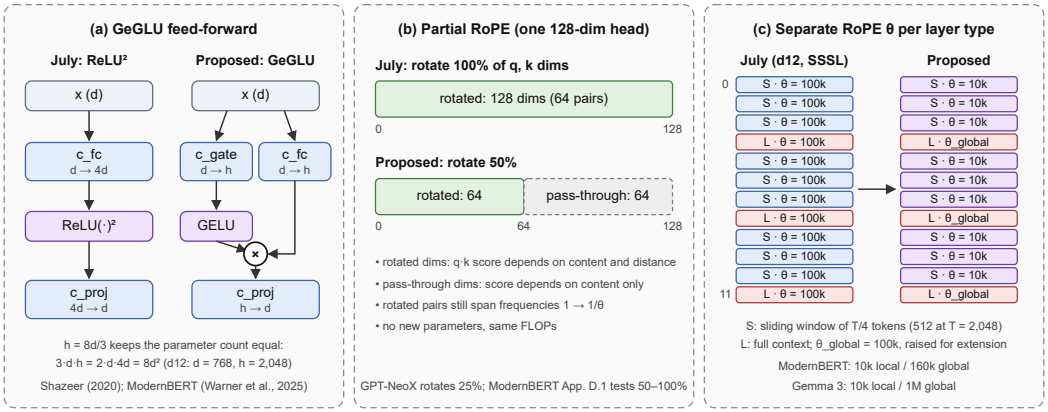


Figure 2: The three proposed changes: (a) GeGLU MLP, (b) partial RoPE, (c) separate RoPE θ for sliding-window (S) and full-context (L) layers.

Table 2: Proposed architecture changes relative to July 2026 nanochat (d = model width, T = context length).

Change	Motivation	Technical Details	Potential Impact
1. GeGLU MLP [16]	In T5, gated MLPs reached lower perplexity than ReLU and GELU MLPs with the same parameter count [16]. PaLM [17], LLaMA [18] and ModernBERT all use one.	$W_{\text{proj}}\big(\text{GELU}(W_{\text{gate}}x) \odot W_{\text{fc}}x\big)$ with $h = 8d/3$, so parameters match ReLU^2 ($h = 2{,}048$ at d12). Flag `--mlp-type=geglu`.	Same parameters and FLOPs. Expect slightly lower loss. ReLU^2 is a strong baseline, so a tie is possible.
2. Partial RoPE [19]	Full RoPE mixes position into every dimension, while unrotated dims can match on content at any distance. GPT-NeoX rotates 25% of each head [19]. ModernBERT found 50–75% slightly better than 100% but kept 100% because its ablations ran at a much smaller scale than the final model.	Rotate the first 64 of the 128 head dims and pass the rest through unchanged. Flag `--rope-frac=0.5`.	No new parameters or FLOPs. Small or no gain at picochat scale; may help context extension.
3. Separate RoPE θ for local and global layers [20]	July uses $\theta = 100{,}000$ everywhere, even in sliding-window layers that only see $T/4$ tokens. ModernBERT uses 10k local and 160k global; Gemma 3 uses 10k and 1M [20].	Two cos/sin tables: sliding-window layers use $\theta = 10{,}000$ and full-context layers keep 100,000. Raise only the global θ when extending context.	No new parameters or FLOPs. We expect no effect at a fixed context length; any benefit should appear during context extension.

Part 2: Ablations on a Nano Nanochat

Results

We tracked every run in the team's Weights & Biases (W&B) project:

`https://forge.coreweave.com/wandb/vraj-shah2121-university-of-toronto/nanochat/workspace`

The three full runs are `d12_baseline`, `d12_geglu` and `d12_rope50`. Each logged validation bits per byte (BPB) every 250 steps, training loss, throughput and MFU every 100 steps, and the CORE score at steps 2000 and 2520. CORE was capped at 500 examples per task. Table 3 summarizes the runs, and Figure 3 compares their training curves.

The W&B configs differ only in the flag under test (`--mlp-type=geglu` or `--rope-frac=0.5`) and the run name. All three runs use seed 42 and 2,520 steps, with the same estimated total of 1.0×10^{18} FLOPs. All three start from a validation BPB of 3.16867, because the zero-initialized output projections make the changes invisible at step 0.

Table 3: Picochat (d12) ablation results. For validation BPB, lower is better; for CORE, higher is better. Gaps are relative to the baseline. Throughput and MFU are medians of the logged steps after the first five.

	Baseline	GeGLU	Partial RoPE (50%)
Hardware	8× H200	8× H200	8× H100
Final validation BPB	0.84537	0.84754	0.84505
Gap to baseline	–	+0.00217	−0.00031
Gap (%)	–	+0.26%	−0.04%
CORE at step 2000	0.1570	0.1547	0.1589
Final CORE (step 2520)	0.1734	0.1656	0.1705
Gap to baseline	–	−0.0078	−0.0029
Tasks better / worse (of 22)	–	9 / 13	8 / 13
Training time (min)	5.23	5.45	5.41
Throughput (million tokens/s)	4.33	4.16	4.10
MFU (%)	41.6	39.9	39.4

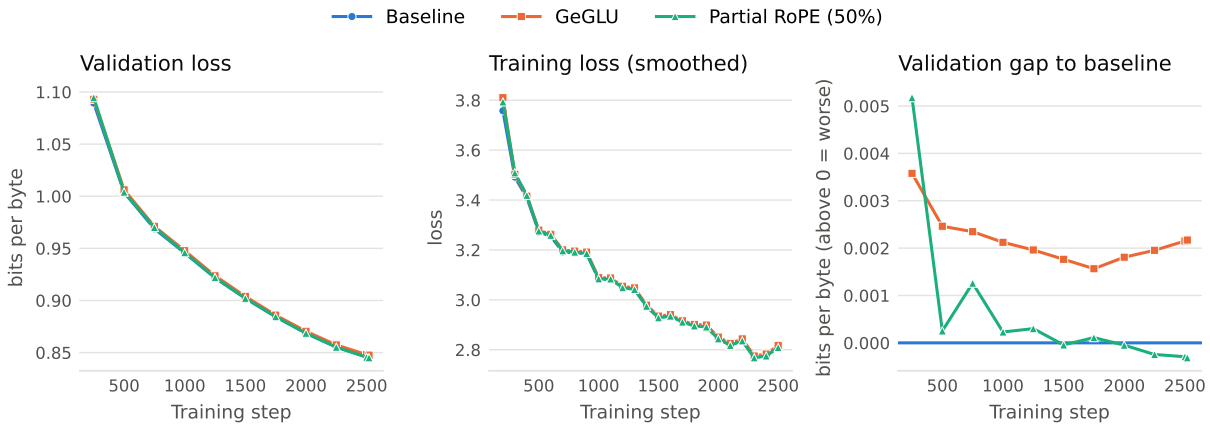


Figure 3: Training curves from the W&B runs. The validation and training curves almost overlap, so the right panel plots each change's validation gap to the baseline. The plots start after the steep early drop (step 250 for validation, step 200 for training loss).

Impact of Each Change

Overall. Neither change clearly improved the model at this scale, which fits the small or null effect we predicted in Part 1. Figure 4 shows why the CORE gaps are weak evidence: individual tasks move by as much as 0.10, in both directions. The standard error of the mean gap across tasks is about 0.008, so GeGLU's gap is 0.95 standard errors from zero and partial RoPE's is 0.34.

GeGLU. GeGLU has a slightly worse validation BPB than the baseline at all 11 evaluations after step 0, with a gap between 0.0016 and 0.0036 that had not closed by the end of training. Its CORE score is also lower, though within the task-level noise above. These evaluations come from one training run and are correlated, so the consistent sign is not 11 independent results. On the same GPU model as the baseline, GeGLU ran about 4% slower in both throughput and training time. Its parameter count and estimated FLOPs match the baseline, so this slowdown is not captured by the FLOP estimate. We do not see the gain reported for gated MLPs in the literature [16], but this run cannot rule one out: it is short and small, and it has only one seed.

Partial RoPE (50%). We cannot distinguish partial RoPE from the baseline. It trails early (a BPB gap of +0.0052 at step 250), catches up by about step 1500, and finishes 0.00031 ahead. It also led on CORE at step 2000 (+0.0019) but trailed at step 2520 (−0.0029), and that change of sign suggests the gap is mostly noise. ModernBERT reported a slight gain at 50–75% [14], and we neither confirm nor contradict that here. We also cannot read its throughput as a cost of the change: this run used H100s while the baseline used H200s, and the change rotates fewer dimensions, so it should not add work.

Limitations. We trained one model per configuration, so we have no direct measure of seed noise. The closest evidence is two identical 100-step smoke tests with the same seed, whose validation BPB differed by 0.00014. That shows only that runs are not fully deterministic, and it is a lower bound on noise, since changing the seed should vary results more. The GeGLU gap is about 15 times that figure and the partial RoPE gap about 2 times. The hardware also differs across runs, as noted above. It should not affect the loss curves much, but it does limit the throughput comparison to baseline versus GeGLU.

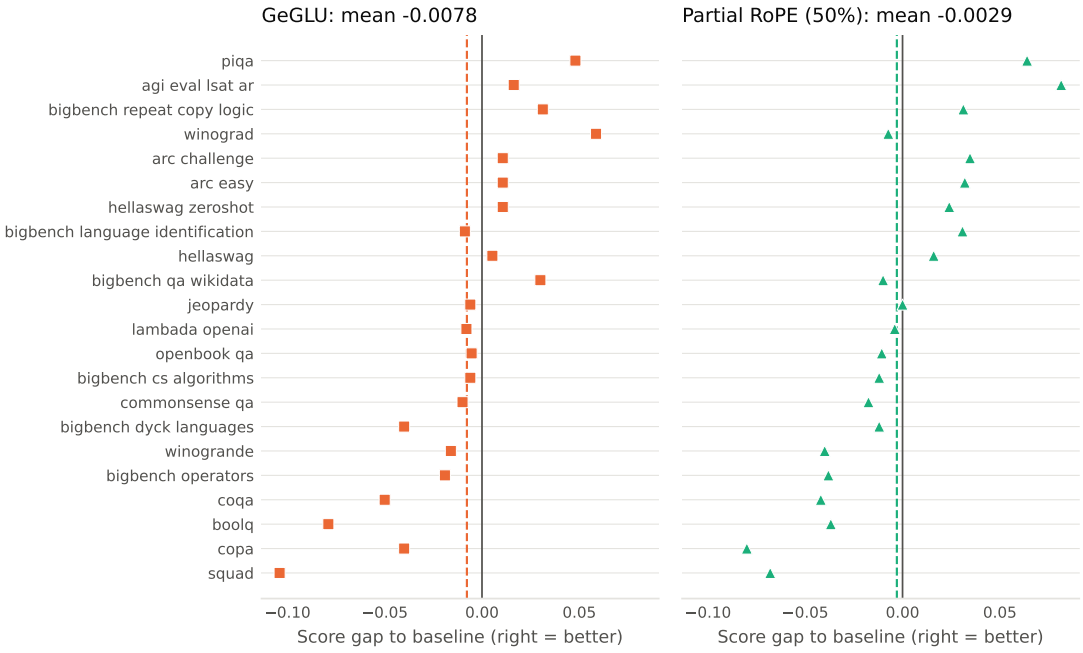


Figure 4: Final CORE gap to the baseline on each of the 22 tasks. The solid line marks zero and the dashed line marks the mean gap. Tasks are sorted by their average gap across the two changes. Both changes gain on some tasks and lose on others.

Part 3: Hardware Efficiency Curves

Part 4: Extending the Context Window

Part 5: Training Our Final Nanochat

Final Model Configuration

Table 4 lists our final nanochat's data, architecture and hardware, with the reason for each choice. The reasons come mostly from our picochat runs in Parts 2 and 4, and several choices keep the final model directly comparable with the d12 picochat.

Table 4: Final nanochat configuration.

Configuration	Choice	Justification
Data	ClimbMix with the same 32,768-token tokenizer as the picochat; 12 tokens per parameter (about 5.2B tokens); 2,048-token context throughout	Same data and tokenizer as the picochat, so the two models compare directly. 12 tokens per parameter is the compute-optimal ratio nanochat measured for its own architecture, and our picochat used the same ratio. We read "the full pre-training dataset" as this full compute-optimal budget, since all 400B tokens would take about 77 times the compute. In Part 4, training at 512 tokens first lowered CORE (0.162 vs 0.173) for only about a 7% speedup.
Architecture	d20 (20 layers, width 1,280) with partial RoPE (50%)	d20 is the original nanochat size and needs about 15 times the picochat's compute. In Part 2, partial RoPE matched the baseline (validation BPB 0.84505 vs 0.84537) at no extra cost, while GeGLU was worse on both loss and CORE, so we leave GeGLU out. We did not implement separate RoPE θ values.
Hardware	8 × H200 in bf16	A 20-step smoke test of d20 on 8 H200s peaked at about 72 GiB per GPU (d12 used 27 GiB). That would leave under 8 GiB of headroom on an 80 GB H100, while the H200's 141 GB runs the full batch with room to spare. The smoke test also reached 48% MFU, above the 41.6% of our d12 runs, and the H200 matches the picochat baseline's hardware and precision.

Training. We launched the run through our Modal script with `--depth=20 --rope-frac=0.5`, saved a checkpoint every 1,000 steps, and left every other setting at its default. For d20, nanochat's batch-size rule picked about 1M tokens per step, twice the d12 batch, and scaled the learning rates up by $\sqrt{2}$ to match. Our first attempt stopped at step 1,920 of 4,980 with no checkpoint saved, most likely because the Modal client on our laptop shut down. We relaunched the run so that it no longer depended on the laptop, and added the checkpoints in case it stopped again. The completed run took 64.7 minutes for 4,980 steps and 5.22B tokens (1.51×10^{19} FLOPs), averaging 1.34M tokens per second and 49.0% MFU. It cost $52.08 in Modal credits including startup and evaluation. Part 5 as a whole cost $155.76, which also covers the failed attempt, the smoke test, the scaling-law runs below, and a d20 baseline and a second-seed picochat that we used to judge partial RoPE (see Results). All of these runs are logged in the same W&B project as Part 2 (`d20_final_v2`, `d20_baseline`, `d12_rope50_seed1337` and the `d*_rope50_scaling` runs). The final model reached

a validation BPB of 0.7367 and a CORE score of 0.242, against 0.8454 and 0.173 for the d12 baseline. Scored on every CORE example instead of 500 per task, it reaches 0.239, while GPT-2 (1.6B) scores 0.257 [1].

Scaling Law: Predicted vs Actual

To predict the nanochat's loss from cheap runs, we trained three more picochats (d8, d10 and d14) with the final model's recipe: partial RoPE, 12 tokens per scaling parameter, ClimbMix and the same tokenizer. With the d12 partial RoPE run from Part 2, they cover a 15-fold range of compute (Table 5). The eval dataset is ClimbMix's held-out validation shard, and every run reports its BPB on the same 42M validation tokens at the end of training. Following Kaplan et al. [21] and Hoffmann et al. [22], we model the loss as a power law in training compute, $\text{BPB} = a\,C^{-b}$, a straight line in log-log space, and fit it by least squares to the four picochats. Some picochat runs finished after the d20, so we fitted the law afterwards, but the d20 itself is not part of the fit.

Table 5: Runs used for the scaling law. All use partial RoPE and 12 tokens per scaling parameter. The d20 was not used in the fit. Training times are on 8 H200s, except d12, which ran on 8 H100s in Part 2.

Model	Scaling params	Tokens	FLOPs	Val BPB	CORE	Train time
d8	41.9M	0.50B	1.49×10^{17}	0.9416	0.105	1.4 min
d10	70.1M	0.84B	4.17×10^{17}	0.8926	0.122	2.3 min
d12	110.1M	1.32B	1.00×10^{18}	0.8451	0.171	5.4 min
d14	164.2M	1.97B	2.22×10^{18}	0.8095	0.181	10.8 min
d20 (predicted)				0.7265		
d20 (actual)	435.2M	5.22B	1.51×10^{19}	0.7367	0.242	64.7 min

The fit is $\text{BPB} = 8.80\,C^{-0.0565}$, and all four picochats lie within 0.33% of it ($R^2 = 0.999$ in log space). At the d20's 1.51×10^{19} FLOPs it predicts a validation BPB of 0.7265, and leaving out one picochat at a time and refitting gives predictions from 0.7230 to 0.7278. The d20 reached 0.7367, 1.4% worse than predicted and just outside that range (Figure 5).

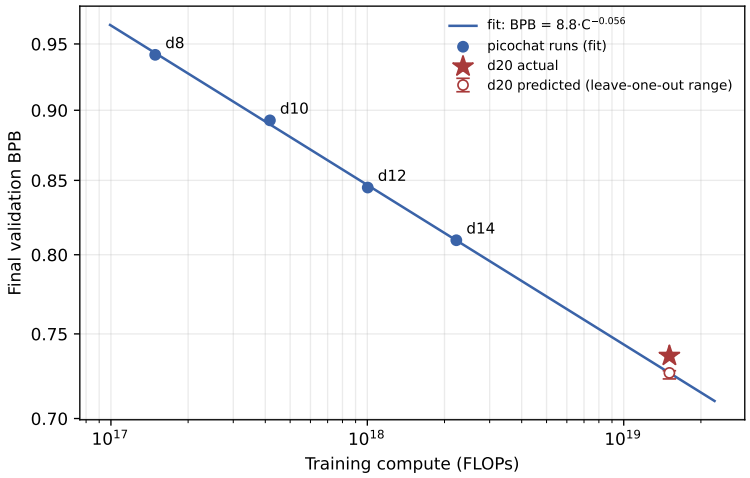


Figure 5: Validation BPB against training compute. The line is fitted to the four picochats; the open circle is its d20 prediction, with the range from leave-one-out refits, and the star is the actual d20.

The prediction was close but too optimistic. The likeliest cause is the form of the law: a pure power law assumes the loss keeps falling at the same rate, while real scaling curves bend toward an irreducible loss. Hoffmann et al. add a constant term for this, $L = E + A\,C^{-\alpha}$ [22], but four points are too few to fit the extra parameter reliably, so our line is probably too steep at large compute. The extrapolation is also long: the d20 used 6.8 times the compute of our largest fitted model, so a small error in the slope becomes a larger error in the prediction. The recipe also changes slightly with size, because nanochat's batch-size rule gave d8 262K tokens per step, d10 to d14 524K and d20 1M, and scaled the learning rates for d8 and d20 to match. Even so, the miss is 0.010 BPB, small next to the 0.073 BPB the model gained from d14 to d20.

We did not fit a power law to CORE. Over these runs it grew roughly linearly with log compute, and a straight-line fit of CORE against $\log_{10} C$ on the four picochats predicts 0.244 for the d20, which reached 0.242.

Results

Table 6 compares the final nanochat with a d20 baseline, trained with full RoPE for this comparison, and with the two d12 picochats from Part 2.

Table 6: Picochat and nanochat results. CORE uses 500 examples per task, as in Part 2. Prompts correct is the number of our 99 completion prompts answered correctly after manual review (see Emergent Abilities).

	Picochat (d12)		Nanochat (d20)	
	Baseline	Partial RoPE	Baseline	Partial RoPE (final)
Width	768	768	1,280	1,280
Scaling parameters	110.1M	110.1M	435.2M	435.2M
Training tokens	1.32B	1.32B	5.22B	5.22B
Training FLOPs	1.00×10^{18}	1.00×10^{18}	1.51×10^{19}	1.51×10^{19}
Hardware	8× H200	8× H100	8× H200	8× H200
Batch size (tokens)	524K	524K	1.05M	1.05M
Training time (min)	5.2	5.4	65.0	64.7
MFU (%)	41.6	39.4	48.7	49.0
Final validation BPB	0.8454	0.8451	0.7371	0.7367
Final CORE	0.173	0.171	0.235	0.242
Prompts correct (of 99)	–	49	–	72

Scale. The partial RoPE picochat and the nanochat share an architecture and differ in size, so the gap between them measures scale. Going from d12 to d20 multiplied training compute by 15. Validation BPB fell by 12.8% (0.845 to 0.737), CORE rose from 0.171 to 0.242, and the nanochat answered 23 more of our 99 prompts correctly. MFU rose too, from about 40% to 49%, most likely because d20's larger matrix multiplications keep the GPUs busier.

Partial RoPE. We kept partial RoPE because it matched the baseline at d12 in Part 2. To measure it at d20, we also trained a d20 baseline with full RoPE and otherwise identical settings. Partial RoPE finished slightly ahead, by 0.0004 BPB (0.7367 vs 0.7371) and 0.007 CORE (0.242 vs 0.235). Its training curve repeated the d12 pattern from Part 2: it trailed early, by 0.006 BPB at step 250, caught up by step 500, and was ahead at 15 of its 20 evaluations, with every gap after step 500 under 0.001 in either direction. To judge gaps this small, we retrained the d12 partial RoPE picochat with a second seed (1337 instead of 42), which changes only the weight initialization. The two seeds differed by 0.0008 BPB and 0.013 CORE, more than either d20 gap. One pair of seeds at d12 is only a rough estimate of the noise, but by that estimate partial RoPE is still indistinguishable from the baseline at d20, as it was at d12.

Excluded changes. GeGLU, slightly worse at d12, and Part 4's short-context recipe, which lowered CORE, are not in the final model. The nanochat's gains therefore come from scale on the July architecture with partial RoPE.

Emergent Abilities

Wei et al. call an ability emergent if it is absent in smaller models but present in larger ones [23]. To look for such abilities, we gave the d12 partial RoPE picochat and the d20 nanochat the same 99 short prompts on geography, science, history, language and arithmetic. Both are base models without chat finetuning, so each prompt is the start of a sentence, such as "The capital of Japan is", and each model continues it for 12 tokens with greedy decoding. A script marked a completion correct if an accepted answer appeared in its first six words; we then read every completion and corrected three of its verdicts. After that review the picochat answered 49 prompts correctly and the nanochat 72. The nanochat was right and the picochat wrong on 24 prompts, and the reverse happened once. Table 7 lists 10 of the 24.

Table 7: Ten prompts the nanochat answers correctly and the picochat does not (greedy completions, trimmed).

Prompt	**Picochat (d12)**	**Nanochat (d20)**
1. The capital of Japan is	Hokkaido, which is the capital of the	Tokyo.
2. The capital of India is	Karnataka, which is the capital of the country	New Delhi.
3. The Mona Lisa was painted by	the French artist Pierre de Coubertin in 17	Leonardo da Vinci in 1503.
4. The freezing point of water in degrees Celsius is	32 degrees Celsius.	0 degrees Celsius.
5. The powerhouse of the cell is the	nucleus.	mitochondria.
6. The hardest natural substance on Earth is	water.	diamond.
7. The first person to walk on the Moon was	a man named Apollo 11, who was born in	Neil Armstrong, who walked on the Moon on July
8. The telephone was invented by	the French in 1860	Alexander Graham Bell in 1876.
9. The plural of mouse is	mouse.	mice.
10. The main language spoken in Brazil is	the native language of the country.	Portuguese.

Most of what the nanochat gained is factual recall: capitals, scientists, inventors and basic science that the picochat gets wrong or avoids. Arithmetic is still out of reach for both. The picochat got none of the 10 arithmetic prompts right and the nanochat got two, "2 + 2 =" and the smallest prime number, while completing "7 + 5 =" with 10. The nanochat also still makes factual errors, such as naming Sydney as the capital of Australia. The one prompt only the picochat got right asked for the name of Martin Luther King Jr.'s "I Have a Dream" speech.

We would not call these abilities emergent in the sense of a sudden jump. CORE rose steadily with compute and landed on the line predicted from the picochats, so the nanochat's gains look like a smooth trend continuing. Schaeffer et al. [24] argue that many reported emergent abilities come from all-or-nothing metrics: a question scored only right or wrong seems to switch on at some scale, even when the probability of the right answer grows smoothly. Our prompt checks are that kind of metric, so Table 7 makes these abilities look more sudden than CORE suggests.

References

[1] A. Karpathy, "nanochat: A tiny chatbot arena and training harness," GitHub, 2025. https://github.com/karpathy/nanochat/discussions/481

[2] A. Vaswani et al., "Attention Is All You Need," *NeurIPS*, 2017. arXiv:1706.03762.

[3] A. Radford et al., "Language Models are Unsupervised Multitask Learners," OpenAI technical report, 2019.

[4] J. Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," *Neurocomputing*, vol. 568, 2024. arXiv:2104.09864.

[5] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," *NeurIPS*, 2019. arXiv:1910.07467.

[6] A. Henry et al., "Query-Key Normalization for Transformers," *Findings of EMNLP*, 2020. arXiv:2010.04245.

[7] D. R. So et al., "Primer: Searching for Efficient Transformers for Language Modeling," *NeurIPS*, 2021. arXiv:2109.08668.

[8] Gemma Team, "Gemma 2: Improving Open Language Models at a Practical Size," arXiv:2408.00118, 2024.

[9] K. Jordan et al., "Muon: An optimizer for hidden layers in neural networks," 2024. https://kellerjordan.github.io/posts/muon/

[10] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *ICLR*, 2019. arXiv:1711.05101.

[11] K. Jordan et al., "modded-nanogpt: Speedrunning the NanoGPT baseline," GitHub, 2024. https://github.com/KellerJordan/modded-nanogpt

[12] J. Shah et al., "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision," arXiv:2407.08608, 2024.

[13] Z. Zhou et al., "Value Residual Learning For Alleviating Attention Concentration In Transformers," arXiv:2410.17897, 2024.

[14] B. Warner et al., "Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference," *ACL*, pp. 2526–2547, 2025. https://aclanthology.org/2025.acl-long.127/

[15] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," arXiv:1810.04805, 2018.

[16] N. Shazeer, "GLU Variants Improve Transformer," arXiv:2002.05202, 2020.

[17] A. Chowdhery et al., "PaLM: Scaling Language Modeling with Pathways," *JMLR*, vol. 24, 2023. arXiv:2204.02311.

[18] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," arXiv:2302.13971, 2023.

[19] S. Black et al., "GPT-NeoX-20B: An Open-Source Autoregressive Language Model," *BigScience Workshop on Challenges & Perspectives in Creating Large Language Models*, 2022. arXiv:2204.06745.

[20] Gemma Team, "Gemma 3 Technical Report," arXiv:2503.19786, 2025.

[21] J. Kaplan et al., "Scaling Laws for Neural Language Models," arXiv:2001.08361, 2020.

[22] J. Hoffmann et al., "Training Compute-Optimal Large Language Models," *NeurIPS*, 2022. arXiv:2203.15556.

[23] J. Wei et al., "Emergent Abilities of Large Language Models," *Transactions on Machine Learning Research*, 2022. arXiv:2206.07682.

[24] R. Schaeffer, B. Miranda, and S. Koyejo, "Are Emergent Abilities of Large Language Models a Mirage?," *NeurIPS*, 2023. arXiv:2304.15004.